

Sandeep Lodhi

Complete Interview Preparation Guide

Node.js Backend Developer (MERN) · Almost 4 Years Experience

Intro · HR · Companies · Projects · Education · Tech Stack Q&A · STAR Stories · Salary & Closing · Cheat Sheet

Contents

1. Start Here and Resume Review
2. Self Introduction
3. HR Questions and Answers
4. Company Wise Experience
5. Project Explanations
6. Education and Personal Background
7. Role and Tech Stack Node Backend QA
8. Experience and Scenario Questions STAR
9. Tricky Questions Salary and Closing
10. Last 30 Minutes Cheat Sheet

00 — Start Here + Final Resume Review

Candidate: Sandeep Lodhi | **Target role:** Node.js Backend Developer (MERN) | **Experience:** almost 4 years (Feb 2023 – today)

This folder is your complete interview kit. It is built from your final resume and everything inside your `about-me` folder (project notes, intros, HR notes, career journey, resume drafts).

1. Files in this kit and the order to read them

#	File	What it gives you	When to read
00	Start Here + Resume Review	Study plan, resume fixes, golden rules	First
01	Self Introduction	30-sec, 1-min, 2-min and backend-focused intros	Day 1, practise daily
02	HR Questions & Answers	45+ HR questions with ready answers	Day 2
03	Company-wise Experience	All 4 companies: what, role, work, why left	Day 2
04	Project Explanations	Every resume project in a simple 5-step format	Day 3
05	Education & Personal	College, school, family, hometown, hobbies	Day 3
06	Role & Tech Stack Q&A	Node.js, Express, DB, Redis, AWS, Docker, security, system design	Day 4–5
07	Experience & Scenario (STAR)	Real stories: challenges, conflict, production bugs, leadership	Day 6
08	Tricky Questions, Salary, Closing	Gaps, job changes, salary, notice period, questions to ask	Day 6
09	Last 30 Minutes Cheat Sheet	One-page revision before the interview	Interview day

2. 7-day study plan

Day	Task	Practise out loud
1	Read 01. Record yourself saying the 1-minute intro on your phone.	Intro 5 times
2	Read 02 and 03. Learn the "why did you leave" line for every company.	Company journey in 2 minutes
3	Read 04 and 05. For each project remember: Problem → What I built → Stack → Challenge → Result.	Explain 3 resume projects
4	Read 06 sections A to E (JavaScript, Node, Express, API, Auth).	20 questions without reading
5	Read 06 sections F to K (Databases, Redis, AWS, Docker, CI/CD, System design).	Draw one architecture on paper

Day	Task	Practise out loud
6	Read 07 and 08. Prepare your salary number and notice period.	5 STAR stories
7	Mock interview with a friend. Then read 09.	Full mock, 45 min

3. Golden rules for speaking in the interview

1. **Speak slowly and simply.** Short sentences are better than long ones. Pause after each point.
2. **Always use this order for any project or story:** Problem → What I did → Tech used → One challenge → Result (with a number).
3. **Use numbers.** 45+ reports, 50 million+ rows, 92 seconds to 300 ms, 30% faster, 40% faster deployment.
4. **Say "I", not "we", for your own work.** Use "we" only for team work.
5. **Only claim what you can explain.** If a skill is on your resume, expect a question on it.
6. **If you don't know, say so honestly.** "I haven't used that in production, but I have used X which solves a similar problem..."
7. **Connect every technical answer to your project.** "In my Hospital CRM, I did this..." sounds much stronger than a textbook answer.

4. Review of your final resume (important — read before applying)

Your final resume is clean, one page, ATS-friendly and well structured. These are the points to fix or prepare for:

Must fix

#	Issue	Why it matters	Suggested fix
1	The file is named "Backend Developer" but the title on the resume says MERN Stack Developer .	Recruiters searching for Node.js Backend roles match on the title.	Change the title to "Node.js Backend Developer (MERN)" and start the summary with backend words.
2	Atiya bullet says "Managed Cloudflare hosting" . Your project notes show deployment on Cloudways (PM2 + Apache).	If the interviewer asks about Cloudflare (DNS, CDN, WAF) and you used Cloudways, it looks wrong.	Write "Cloudways (PM2 + Apache)" or keep Cloudflare only if you really managed Cloudflare DNS/SSL.
3	Atiya bullet says "WebRTC-based video consultation" . Your Atiya project notes do not show WebRTC code.	WebRTC questions are deep (STUN, TURN, signaling, SDP).	Keep it only if you built it. Your astrology project at MSAI had audio/video calls, so you can talk about WebRTC concepts from there. Prepare file 06 section L.
4	Hospital Appointment CRM says "using MERN stack and SQL". That CRM uses SQL Server , not MongoDB.	A sharp interviewer will notice "MERN" with no MongoDB.	Write "Node.js, Express, React and SQL Server".

#	Issue	Why it matters	Suggested fix
5	Old resumes list House of VSB (Sep 2023 – Feb 2024) and MSAI from Mar 2024 . The final resume shows MSAI Aug 2023 – Oct 2024 .	Background verification (BGV) checks offer letters, relieving letters, PF/UAN and Form 16.	Make sure the dates on the final resume match your documents exactly. Keep the same story in every interview.

Strongly recommended for a backend resume

Your best backend achievements are in your Atiya projects but are **missing** from the resume. Add 2–3 of these bullets under Atiya Healthcare:

- Optimised a slow SQL Server report query from **~92 seconds to ~300 ms** by bypassing a per-row view and adding targeted indexes.
- Built a BI reporting platform with **45+ reports** over **50M+ row** PostgreSQL and SQL Server tables, using an ETL sync into a local read-model, Redis caching and BullMQ workers.
- Built a cron-based automation service that detects dropped calls every 30 minutes and pushes them to dialer campaigns for auto call-back, with dry-run and re-entrancy guards.
- Designed a 3-tier cache (in-memory LRU → Redis → SQLite → PostgreSQL) that cut patient call-history lookups from minutes to milliseconds.
- Secured APIs with JWT, email OTP two-factor login, permission-based RBAC, Joi validation, Helmet and rate limiting.

Suggested backend summary (copy-paste)

Node.js Backend Developer with almost 4 years of experience building scalable REST APIs, background jobs and data-heavy systems with Node.js, Express, MongoDB, PostgreSQL, SQL Server and Redis. Experienced in query optimisation (92 s → 300 ms), queues (BullMQ, RabbitMQ), real-time systems (Socket.IO), serverless (AWS Lambda) and production deployment (Docker, PM2, NGINX, IIS, CI/CD). Delivered projects for clients in Israel, Spain, USA and Australia.

Other small points

- You have two resume files (3+ years and 4 years) that are almost identical. Keep one so you don't send the wrong one.
- Experience from Feb 2023 to Oct 2026 is about **3 years 8 months**. "Almost 4 years" or "3.8 years" is safest in the interview. Don't say "more than 4 years".
- Skills like **Nest.js, Prisma, Microservices, OAuth, Kubernetes** are listed. Prepare at least the basic questions for each (file 06 has them).
- TorahAnytime stack on the resume says MySQL, Redis, AWS, Docker. Your notes also mention RabbitMQ, tRPC, ClickHouse, RethinkDB, DigitalOcean, Kubernetes. Be ready to explain where each was used, or don't mention it.
- Your notes put the **PDF editor** under Perk in some files and ShowTrail in others. Pick the correct one and keep it the same everywhere.

5. Your story in one line (memorise)

"I'm a Node.js backend developer with almost 4 years of experience. I started with SQL and jQuery at Shaligram, built full MERN products at MSAI, learned AWS serverless and DevOps at Morpheme while leading an Israeli client project, and now at Atiya Healthcare I build and deploy backend systems for a large healthcare call centre — CRM, WhatsApp automation, reporting and automation jobs."

01 — Self Introduction ("Tell me about yourself")

This is the first question in almost every interview. It sets the mood for the whole interview. Practise it until it sounds natural, not memorised.

Simple formula: Who I am → Experience summary → Current work → Key strengths → What I want next.

1. Technical round — 1 minute (use this most often)

"Good morning. Thank you for this opportunity.

My name is Sandeep Lodhi. I'm a Node.js backend developer with almost 4 years of experience in building REST APIs, backend services and full-stack MERN applications.

Currently I'm working at **Atiya Healthcare** in Delhi. There I build backend systems for a large healthcare call centre — a Hospital Appointment CRM, a WhatsApp automation platform using Exotel APIs, reporting dashboards and automation jobs. I work with Node.js, Express, SQL Server, PostgreSQL, MongoDB and Redis, and I also handle deployment on IIS and Cloudways.

Before this, at **Morpheme Webnexus**, I worked on AWS Lambda serverless backends and CI/CD, and I led an Israeli client project called **TorahAnytime**, where I reviewed and merged PRs and deployed to production.

My strengths are API design, database query optimisation and production debugging. For example, I reduced a slow report query from about 92 seconds to around 300 milliseconds.

Now I'm looking for a backend-focused role where I can work on scalable systems and grow as a Node.js engineer. That's a short summary about me."

2. Short version — 30 seconds (telephonic / screening round)

"Hi, I'm Sandeep Lodhi, a Node.js backend developer with almost 4 years of experience. I work mainly with Node.js, Express, MongoDB, SQL databases and Redis. Currently at Atiya Healthcare I build CRM and WhatsApp automation backends for a healthcare call centre. Earlier I worked on AWS serverless projects and led an Israeli client project at Morpheme Webnexus. I'm looking for a backend role where I can build scalable systems."

3. Full career journey — 2 minutes (when they say "walk me through your career")

"My name is Sandeep Lodhi. I completed my B.Tech in Computer Science from Technocrats Institute of Technology, Bhopal, in 2023 with 8.7 CGPA.

I started my career at **Shaligram Infotech, Ahmedabad**, as a software developer. There I built my foundation in JavaScript, jQuery, Bootstrap and MS SQL Server, and I worked on optimising APIs and SQL queries.

Then I joined **AB MSAI Research Labs** in Delhi as a MERN stack developer. There I did most of the R&D for an astrology platform from scratch — I chose the tech stack, integrated third-party astrology APIs for kundli and horoscope, and built chat, audio and video communication features. I also built and deployed **ThePNAC.com** end to end using PM2 and NGINX.

After that I joined **Morpheme Webnexus**, where I moved deeper into backend and cloud. I built serverless APIs on AWS Lambda with TypeScript, set up CI/CD with Sentry and Codecov, did load testing with JMeter, and led the **TorahAnytime** project for an Israeli client — assigning tickets, reviewing PRs and deploying to production. In parallel I built **BikeShopShippers**, a logistics platform on Medusa.js with Stripe and UPS integration.

Currently I'm at **Atiya Healthcare**, where I own backend systems end to end — the Hospital Appointment CRM, a real-time WhatsApp platform with Socket.IO, reporting on tables with 50 million+ rows, and automation jobs. I gather requirements, design the APIs and database queries, build, deploy and support them in production.

I enjoy backend engineering the most — performance, clean architecture and solving real business problems. That's why I'm looking for a backend-focused Node.js role now."

4. HR round version (more personal, less technical)

"Thank you for giving me this opportunity to introduce myself.

My name is Sandeep Lodhi. I'm from Ashok Nagar in Madhya Pradesh, and I come from a farming family. I completed my schooling in Lalitpur and my B.Tech in Computer Science from Technocrats Institute of Technology, Bhopal, with 8.7 CGPA.

I have almost 4 years of experience as a software developer, mainly in Node.js backend and the MERN stack. I've worked with 4 companies and with clients from Israel, Spain, the USA and Australia. Currently I'm working at Atiya Healthcare in Delhi.

As a person, I'm hard-working, I take ownership of my work and I enjoy learning new technologies. In my free time I like playing cricket and guiding students who are starting in programming.

I'm looking for a stable, long-term role where I can grow and contribute to the company. Thank you."

5. Backend / product-company version (strong technical focus)

"I'm Sandeep, a backend developer focused on Node.js. Over almost 4 years I've built REST APIs, background job systems, real-time messaging and data-heavy reporting.

A few things I've built recently: a WhatsApp platform where Exotel webhooks are stored in MongoDB and pushed to agents in real time with Socket.IO; a reporting platform with 45+ reports on tables with 50 million+ rows, using ETL sync, Redis caching and BullMQ workers; and a cron service with safety guards that pushes dropped calls back into dialer campaigns.

I care about performance and reliability — indexing, caching, retries with backoff, idempotent webhooks, health checks and graceful shutdown. I'd like to bring this to a team building scalable products."

6. Tips for speaking

Do	Don't
Smile and keep eye contact (look at the camera on video calls).	Don't read from the screen.
Speak at a medium speed. Pause between companies.	Don't rush to finish in 20 seconds.
Mention 1 number (92 s → 300 ms, 45+ reports).	Don't list 25 technologies in one breath.
End with what you want next.	Don't say anything negative about your current company.
Keep it 60–90 seconds unless they ask for detail.	Don't tell your full family story in a technical round.

After your intro, the interviewer usually picks one thing you said. So only say things you are ready to explain — Atiya CRM, WhatsApp platform, TorahAnytime, AWS Lambda and query optimisation.

7. Common follow-ups right after the intro

"Which project are you most proud of?"

"The Hospital Appointment CRM at Atiya. I replaced an old ASP.NET system with Node.js and React on the same SQL Server databases, with 47 API modules and integrations with the dialer, WhatsApp and SMS. It's used daily by call-centre agents."

"What is your current role exactly?"

"I'm a MERN stack developer, but mostly backend. I own the full lifecycle — requirements, API and database design, development, deployment on IIS and Cloudways, and production support."

"Why backend and not frontend?"

"I enjoy problems where data, performance and reliability matter — like making a 92-second query run in 300 ms. I can do React well, but backend is where I add the most value."

02 — HR Questions and Answers

HR wants to know three things: **Can you do the job? Will you stay? Will you fit in the team?** Every answer should support one of these.

Keep each answer 30–60 seconds. Be positive. Never criticise a past company or manager.

A. About you

1. Tell me about yourself. Use the HR version in file 01, section 4.

2. Describe yourself in three words.

"Responsible, hard-working and curious. Responsible because I own my work till production. Hard-working because I come from a farming family where I saw daily effort. Curious because I keep learning — from jQuery to AWS Lambda to queues and caching."

3. What are your strengths?

"My first strength is ownership. At Atiya I handle features end to end — from requirement discussion to deployment and production support. My second strength is problem solving, especially performance problems. I brought a report query down from about 92 seconds to around 300 milliseconds. My third strength is adaptability. I've worked on healthcare, logistics, edtech, media and astrology projects, and with clients from four countries."

4. What is your weakness?

"Earlier I used to take on too many tasks at once because I didn't like saying no. Sometimes it put pressure on deadlines. Now I break work into smaller tasks, give realistic timelines, and tell my manager early if something is at risk. It has improved my planning a lot."

Other safe options: "I sometimes spend extra time polishing code — now I time-box it." / "Public speaking — I'm improving by doing demos to stakeholders."

5. How would your friends or colleagues describe you?

"As a hard-working and helpful person. Colleagues come to me for debugging help and deployment issues. Friends say I'm calm and fun-loving."

6. What motivates you?

"Seeing my work used by real people. When call-centre agents use my CRM every day, or a manager gets a report in seconds instead of minutes, that motivates me."

7. What are your hobbies?

"I play cricket, and I like guiding students who are starting in programming. I also like exploring new tech tools and reading about system design."

B. About the job and company

8. Why do you want to join our company? Research the company first (website, LinkedIn, Glassdoor, product). Then use this template:

"I read that your company works on [product/domain]. Your stack — Node.js, [MongoDB/PostgreSQL/AWS] — matches my experience. I'm especially interested because [scale/product/clients]. I want to work in a strong engineering team on a product with real users, and this role is backend-focused, which is exactly where I want to grow."

9. Why should we hire you?

"Three reasons. First, I have almost 4 years of hands-on Node.js backend experience — APIs, databases, queues, caching and deployment. Second, I've owned full projects in production, so I don't need hand-holding — I can take a requirement and ship it. Third, I've worked with international clients and different domains, so I adapt quickly. I'll be productive from the first few weeks."

10. What do you know about our company? Prepare 4 points: what they build, who their customers are, tech stack (from job description), recent news or funding.

11. Why are you looking for a change?

"I've learned a lot at Atiya — I own complete backend systems there. Now I want to work in a larger engineering team on a product at scale, with stronger code reviews and system design challenges. This role is backend-focused, which matches my long-term goal."

12. Where do you see yourself in 5 years?

"In 5 years I see myself as a senior backend engineer or tech lead — someone who designs systems, mentors juniors and owns important services. I want to grow in system design, distributed systems and cloud. I'd like to do that growth within one company for the long term."

13. What are your short-term and long-term goals?

"Short-term: join a good team, understand the product quickly and start delivering quality backend features. Long-term: become a strong backend architect who can design scalable systems and lead a team."

14. How long will you stay with us?

"I'm looking for a long-term role. As long as I'm learning, growing and contributing, I don't see a reason to move. My goal is to grow into a senior role in the same company."

15. What kind of work environment do you like?

"A team where people share knowledge, code is reviewed, and there is clear ownership. I'm comfortable in both fast-paced and structured environments."

C. Working style and pressure

16. Can you work under pressure?

"Yes. In production support at Atiya, when an issue affects agents during live calls, I have to fix it quickly. I stay calm, check logs first, find the root cause, apply a safe fix, and then document it. For example, when new code was not loading on IIS after deployment, I found that iisnode was not watching the source files, fixed the config and documented the restart steps."

17. Are you a team player?

"Yes. At Morpheme I worked with cross-functional teams, and on TorahAnytime I coordinated developers — assigning tickets, reviewing PRs and helping them when they got stuck. Even at Atiya, where I'm the main developer, I work closely with call-centre managers, QA and the IT team."

18. Can you work independently?

"Yes, that's one of my strengths. At Atiya I built 8 applications mostly on my own — from requirements to deployment. I also built ThePNAC.com and BikeShopShippers end to end."

19. Are you comfortable working late or on weekends?

"Yes, when the work needs it — like a production issue or a release. I plan my work to avoid it regularly, but I'm committed when it's important."

20. Are you willing to relocate?

"Yes, I'm open to relocation. I've already worked in Ahmedabad, Delhi and Greater Noida." *(Change if you are not open.)*

21. Remote, hybrid or office?

"I'm comfortable with all three. I've worked remotely with international clients using Slack, Jira and Zoom, and I'm also comfortable in the office."

22. How do you handle criticism?

"I take it positively. In code reviews, feedback helped me write cleaner code. I listen, ask questions if something is unclear, and improve. I don't take it personally."

23. How do you handle a conflict with a colleague?

"I talk directly and calmly, focus on the problem and not the person, and use data. On one project there was a disagreement about retrying a dialer API automatically. I explained the risk — a retry could call a customer twice — and proposed a safer option. We agreed on it and I documented the decision."

24. How do you prioritise when you have many tasks?

"First production issues, then deadline-based tasks, then improvements. I list tasks in Jira or a simple note, estimate them, and agree priorities with my manager when two things clash."

25. How do you keep yourself updated?

"I follow Node.js release notes, read engineering blogs, watch system design videos and try new tools in small projects. For example, I learned BullMQ and Redis caching by using them in real projects."

D. Achievements and failures

26. What is your biggest achievement?

"Replacing the legacy ASP.NET CRM at Atiya with a Node.js and React system without changing the database. It had 460+ tables. I wrote a script that generated 463 model files from the SQL schema, which saved weeks of work. Today agents use it daily inside the dialer."

27. Tell me about a failure.

"In one of my early projects, I kept environment files with credentials inside the repository to move faster. Later I realised it was a security risk. I learned from it — now I keep only a `.env.example` in git, inject secrets on the server, and rotate keys if they were exposed."

28. Tell me about a mistake you learned from.

"Once I built a reporting feature that read data from a child table. Results were always empty for the last 30 minutes. I found that the table was filled by an hourly ETL job. I learned to always check how and when the source data is written before building on it. I switched to the live parent table and documented it."

29. Have you ever led a team?

"Yes, on TorahAnytime at Morpheme. I assigned tickets, reviewed and merged pull requests, guided developers and handled production deployments for the Israeli client."

E. Offer and joining

30. What is your current CTC?

"My current CTC is [X] LPA fixed." (Say the exact number. It will be checked with salary slips.)

31. What is your expected CTC?

"Based on my almost 4 years of experience, my backend skills and the market for this role, I'm expecting [Y] LPA. I'm open to discussion based on the overall offer and growth." *Tip: research the range on Glassdoor/AmbitionBox for "Node.js developer 4 years" in your city. A 30–50% hike over current is normal for a switch.*

32. What is your notice period?

"My notice period is [30/60/90] days. I can try to get early release if needed."

33. Do you have any other offers?

If yes: "Yes, I have one offer, but this role is my first preference because [reason]." If no: "I'm in process with a few companies, but I'm most interested in this role."

34. Why did you change companies so often? → See file 08, it has a full answer.

35. Are you overqualified for this role?

"I don't think so. I have the right experience to start contributing quickly, and there's still a lot I want to learn here, like [their tech/scale]."

F. Classic "thinking" questions

36. What is the difference between hard work and smart work?

"Hard work is putting in effort and time. Smart work is choosing the right method so the effort gives more results. For example, instead of hand-writing 463 model files, I wrote a script to generate them. Best results come from both together."

37. What is the difference between confidence and overconfidence?

"Confidence is believing in yourself based on preparation. Overconfidence is believing you're right without preparation or ignoring risks. Confidence says 'I can do this'; overconfidence says 'I don't need to test this'."

38. What is success for you?

"Success is when my work solves a real problem and people use it. Personally, it's growing every year and supporting my family."

39. Who inspires you?

"My father. He's a farmer and works hard every day without complaining. He taught me discipline and patience."

40. What makes you angry?

"I don't get angry easily. What disappoints me is careless work that breaks production — so I prefer clear processes and testing."

41. Would you lie for the company?

"No. I'll always be loyal to the company, but honesty is important. I'll look for an honest solution that also protects the company."

42. If you won a lottery, would you still work?

"Yes. I enjoy building software, and work gives me learning and purpose, not just money."

43. What if we reject you?

"I'll take it as feedback, ask where I can improve, and work on it. I'd also like to apply again in the future."

44. Rate yourself in Node.js out of 10.

"I'd say 7.5 out of 10. I'm strong in APIs, databases, async patterns, queues and deployment. I want to go deeper in Node internals and large distributed systems — that's what I'm working on."

45. Do you have any questions for us? — Always say YES. See file 08, section D.

03 — Company-wise Experience (what, role, work, why left)

For each company, remember 5 things: **What the company does** → **My role** → **What I worked on** → **What I learned** → **Why I moved**.

Company	Role	Dates (as on final resume)	Location	One-line summary
Atiya Healthcare Pvt. Ltd.	MERN Stack Developer (backend focus)	Aug 2025 – Present	Karol Bagh, Delhi	Healthcare call-centre CRM, WhatsApp automation, reports, automation jobs
Morpheme Webnexus Pvt. Ltd.	MERN Stack Developer	Nov 2024 – Jul 2025	Greater Noida	AWS Lambda serverless, CI/CD, led TorahAnytime, built BikeShopShippers
AB MSAI Research Labs Pvt. Ltd.	MERN Stack Developer	Aug 2023 – Oct 2024	New Delhi	Astrology platform R&D, ThePNAC.com end to end, US "OD" project
Shaligram Infotech	Software Developer	Feb 2023 – Aug 2023	Ahmedabad	Foundation: JavaScript, jQuery, Bootstrap, MS SQL Server

BGV warning: these dates must match your offer letters, relieving letters and PF/UAN records. Older resume drafts showed different dates and a "House of VSB" role. Use one consistent story.

1. Atiya Healthcare Pvt. Ltd. (current)

What the company does

"Atiya Healthcare is a healthcare tele-consultation company. It runs clinics, a large call centre using the Ameyo dialer, and online medicine orders with courier delivery. Patients book clinic appointments through call-centre agents."

My role

"I'm a MERN stack developer with a strong backend focus. I'm the main owner of the new Node.js systems — I gather requirements from managers, design APIs and queries, build backend and frontend, deploy, and support in production."

What I worked on

Application	What it does	Key backend work
Hospital Appointment CRM	Agents book and manage clinic appointments inside the dialer. Replaced a legacy ASP.NET app.	Express 5, SQL Server (2 DBs), 47 API modules, Ameyo + WhatsApp + SMS integration, RBAC, IIS deployment

Application	What it does	Key backend work
WhatsApp Automation Platform	Real-time shared WhatsApp inbox for agents	Exotel webhooks → MongoDB → Socket.IO, multi-number routing, retries with backoff, Cloudways + PM2
ViewReports (BI platform)	45+ call-centre reports	ETL sync from 50M+ row tables, Redis, BullMQ worker, JWT + OTP, Excel export guards
QMS (quality monitoring)	Full audit of one call + recording playback	Query from 92 s to 300 ms, audio streaming proxy with HTTP Range
Patient-CRM	Call history of a patient in one search	3-tier cache: LRU → Redis → SQLite → PostgreSQL
CallDropAutoDial	Auto call-back of dropped calls	node-cron every 30 min, DRY_RUN flag, re-entrancy guard

What I learned

"End-to-end ownership, working with large legacy databases, query optimisation, caching strategies, background jobs, and Windows IIS deployment with iisnode."

Why are you looking for a change?

"At Atiya I've owned complete systems, and I'm proud of that. Most of the time I'm the only developer on these codebases. Now I want to work in a bigger engineering team, with code reviews and larger-scale products, so I can grow faster as a backend engineer."

Possible questions - "*How big is the team?*" → "I'm the main developer for these Node.js apps. I work with call-centre managers, QA, the IT team and seniors for decisions." - "*Are these apps live?*" → "Yes. The CRM, reports, QMS and automation jobs run on Windows IIS internally; the WhatsApp platform runs on Cloudways."

2. Morpheme Webnexus Pvt. Ltd.

What the company does

"Morpheme Webnexus is a SaaS and custom web development company. It builds products for international clients in the US, Israel, Spain and Australia."

My role

"MERN stack developer. I started with serverless backend tasks and grew into leading a client project."

What I worked on

Project	Domain	My work
PERK	Event management	Serverless CRUD APIs on AWS Lambda with TypeScript; PDF-to-image conversion; custom PDF editor for event layout maps (stalls, gates, routes); multi-role users
Nosotros	LMS for Spanish schools	Set up CI/CD with Bitbucket Pipelines; integrated Sentry (errors) and Codecov (test coverage); Jira and Slack notifications; worked on tickets
FamilyOne	Family discount network	Optimised Lambda functions; wrote serverless YAML configs for scaling; load tested APIs with JMeter; integrated Sentry and Codecov
ShowTrail	Coupon promotion (same admin as FamilyOne)	Trained Intercom AI workflows for customer support chatbot
SmartBiz	B2B onboarding	APIs and UI for dynamic forms per business sector (finance, medical, toys, technology)
TorahAnytime	Media + donation platform (Israel client)	Led the project: assigned tickets, reviewed and merged PRs, deployed to production; built features like likes, editable playlists, legacy speakers, most-viewed filter
BikeShopShippers	Logistics	Built from scratch on Medusa.js: UPS (rates, labels, tracking), Stripe payments, Google address autocomplete, CI/CD with GitHub Actions

Resume impact line: "Improved deployment efficiency by 40% using AWS Lambda and CI/CD workflows."

How to explain 40%: "Before CI/CD, deployments were manual and took long, with mistakes. After pipelines with automated tests and deploy steps, release time dropped by about 40% and failed releases reduced."

What I learned

"AWS serverless, TypeScript, CI/CD, monitoring with Sentry, load testing, Docker, and leading a project with code reviews."

Why did you leave?

"I learned a lot at Morpheme, especially AWS and DevOps. But most work was short client projects. I wanted to work on long-term products where I own the system end to end. Atiya gave me that — complete ownership of backend systems used daily."

3. AB MSAI Research Labs Pvt. Ltd.

What the company does

"AB MSAI Research Labs is an AI and software solutions company in Delhi that builds web platforms for clients."

My role

"MERN stack developer."

What I worked on - Astrology platform (from scratch): "I did more than 70% of the R&D. I defined the tech stack — backend, frontend, database — and integrated a third-party astrology API (Prokerala) for kundli making, kundli matching, horoscope and baby names. I built chat, audio and video call features for astrologer–user consultation, and built the first prototype of backend and frontend." - **ThePNAC.com:** "An individual project I built from scratch to end — backend, frontend, database, fully responsive — and deployed on the company server using PM2 and NGINX." - **OD (US-based project):** "I supported the team in validating services and setting access permissions for visitor data, and worked on responsive UI."

Resume impact lines: engagement +20% and user experience +30% through responsive UI. *How to explain:* "After the responsive redesign, mobile users stayed longer and bounce rate dropped. The client reported around 20% more engagement." *(Only quote numbers you can support. If asked "how did you measure it?", say "from Google Analytics reports shared by the client/manager".)*

What I learned

"Building a product from zero, choosing a tech stack, third-party API integration, real-time features, and deploying on a Linux server with PM2 and NGINX."

Why did you leave?

"The main project was completed and handed over to the client. After that, the company changed focus and the development team was released. It was a great learning experience."

4. Shaligram Infotech (first job)

What the company does

"Shaligram Infotech is an IT services company in Ahmedabad that builds web applications for clients."

My role

"Software developer — my first job after college."

What I worked on

"I worked on web applications using HTML, CSS, JavaScript, jQuery, Bootstrap and MS SQL Server. I wrote stored procedures and queries, built APIs, and fixed performance issues. I improved some API response times by around 30% and SQL query performance by around 20% using indexes and better queries." "To strengthen my MERN skills, I also built a Real Estate platform and an E-commerce platform using React, Node.js, Express and MongoDB."

What I learned

"Strong basics — SQL, JavaScript, debugging, working in a professional team, and using Git."

Why did you leave?

"After training, the company's requirement changed and they wanted to move me to a non-development role like manual testing and UI work. I wanted to grow as a developer, so I moved to a MERN developer role at MSAI."

5. Common questions about your companies

"Walk me through your career." / "Explain your job switches."

"Shaligram was my first job where I built foundations. I moved because the role was changing away from development. At MSAI I built full products, and moved when the project was handed over to the client. At Morpheme I learned AWS and DevOps and led a client project. I moved to Atiya to own complete products. Every move gave me a bigger responsibility."

"Which company did you learn the most in?"

"Technically, Atiya — because I own everything, from database queries to deployment. For teamwork and cloud, Morpheme."

"Which company did you like the most?"

"Each had something good. Morpheme had great international exposure; Atiya gave me ownership. I liked the learning in both."

"Did you work with international clients?"

"Yes. At Morpheme with clients from Israel (TorahAnytime), Spain (Nosotros LMS), the US and Australia. I attended calls, understood requirements in English, and worked across time zones using Slack, Jira and GitHub."

04 — Project Explanations (simple, speakable, deep enough)

How to explain ANY project — the 5-step formula

1. **Problem:** What business problem existed?
2. **What I built:** The solution in 2–3 lines.
3. **Tech stack and architecture:** Frontend → Backend → Database → Integrations → Deployment.
4. **One challenge:** What was hard and how I solved it.
5. **Result:** Number or impact.

Speak for 1–2 minutes first. Then let the interviewer ask deeper questions.

Before each interview, revise the 3 resume projects (TorahAnytime, Hospital CRM, BikeShopShippers) and the WhatsApp platform. Those are the most likely questions.

PROJECT 1 — Hospital Appointment CRM (Atiya Healthcare) ★ main project

Stack: Node.js, Express 5, React 19, Redux Toolkit, SQL Server (2 databases), Redis, Joi, Winston, Swagger, IIS + iisnode

Spoken explanation (1.5 min)

"At Atiya Healthcare, call-centre agents book clinic appointments for patients. They used an old ASP.NET WebForms CRM that was slow and hard to change.

I rebuilt it as a Node.js + Express backend and a React frontend, on top of the same SQL Server databases, because many other systems depended on those databases.

The backend has 47 API modules — appointments, patients, call dispositions, complaint tickets, consultation forms and courier tracking. It follows a layered structure: routes, Joi validation, controller, service and repository with raw parameterised SQL.

It integrates with the Ameyo dialer — the CRM opens inside the dialer as an iframe and the backend can auto-dispose calls. On booking, it sends a WhatsApp confirmation through Exotel and an SMS through ValueFirst.

The biggest challenge was size — 460+ tables across two databases. I wrote a script that read the SQL schema and generated 463 model files automatically, which saved weeks.

I deployed it on Windows IIS with iisnode, running 4 Node processes, with health endpoints and log rotation. Agents use it every day, and we reduced processing time by around 30%."

Architecture (draw this if asked)

```
Ameyo Dialer → iframe → React SPA (IIS static site)
    ↓ axios (agent headers / JWT for admin)
    Express 5 API on IIS + iisnode (4 processes)
    routes → Joi validation → controller → service → repository (raw SQL)
    ↓
    ↓
SQL Server: CRM DB + Clinic DB    Ameyo API / Exotel WhatsApp / SMS / Courier APIs
Redis: dialer session state
```

Deep-dive Q&A

- **Why raw SQL instead of an ORM?** "The legacy schema had 460+ tables and complex queries. Raw parameterised SQL gave full control over performance. Parameters prevent SQL injection."
- **How did you connect to two databases?** "One Database class with two named connection pools — one for the CRM DB and one for the clinic DB — with a concurrency limit."
- **How does login work inside the dialer?** "The dialer already authenticates the agent. It passes user and session IDs by URL or postMessage. The frontend sends them as headers, and the backend resolves the agent and caches it for 5 minutes. Admin users log in with JWT."
- **What if WhatsApp sending fails?** "It never breaks the booking. I retry only on network errors, 5xx or 429, up to 3 times with exponential backoff. Failures are logged in a table. Delivery status comes back through a webhook protected with a secret token."
- **How did you handle the old ASP.NET passwords?** "I re-implemented the same encryption in Node (PBKDF2 + AES-256-CBC) so existing users didn't need to reset passwords."
- **RBAC?** "Page-level rights from the database — view, download, edit. Middleware checks them on each route."
- **Processing time reduced 30% — how?** "Optimised SQL queries and indexes, fewer round trips per screen, caching of lookups, and connection pooling."

PROJECT 2 — WhatsApp Automation Platform (Atiya Healthcare)

Stack: Node.js, Express, Socket.IO, MongoDB Atlas (Mongoose), React, Exotel WhatsApp Business API, Cloudinary, PM2, Apache on Cloudways

Spoken explanation

"The company talked to customers on two WhatsApp Business numbers, but agents used phones, so there was no shared inbox, history or reports.

I built a real-time WhatsApp inbox. Exotel sends incoming messages and delivery receipts to my Express webhook. I save them in MongoDB and push them instantly to agents' browsers using Socket.IO. Agents reply from the web UI, and the backend automatically sends the reply from the same business number the customer messaged.

It has unread counts, media upload through Cloudinary, campaign/template messages, and admin reports using MongoDB aggregation.

One challenge: a delivery status sometimes arrived before the outgoing message was saved. I fixed it by upserting by the Exotel message ID, so the order doesn't matter. This also makes webhook retries safe — no duplicates.

I deployed it on Cloudways with PM2 and an Apache reverse proxy that also forwards WebSocket traffic."

Deep-dive Q&A - Why MongoDB here? "Messages are document-shaped with different payloads, high write rate and simple access patterns by contact and time." - **Indexes?** "Compound indexes like `{to, isRead}` and `{from, to, isRead}` for unread counts, and `{direction, timestamp}` for lists." - **How will Socket.IO scale to many servers?** "Use the Socket.IO Redis adapter so events reach all instances, plus sticky sessions on the load balancer." - **Webhook security?** "Webhooks are public by nature. API routes use an API key. The improvement is signature verification on webhooks." - **Voice notes rejected by WhatsApp?** "I transcoded audio to MP3 through Cloudinary, which WhatsApp accepts."

PROJECT 3 — TorahAnytime (Morpheme Webnexus, Israeli client) ★ resume project

Stack (resume): Node.js, MySQL, Redis, AWS, Docker. *Also used in the project per your notes:* RabbitMQ, tRPC, DigitalOcean, ClickHouse, RethinkDB, Kubernetes.

What it is: A global Torah lecture (religious media) and donation platform. Speakers register, upload lectures and classes, and run donation campaigns. It serves many concurrent users worldwide.

Spoken explanation

"TorahAnytime is a high-traffic media and donation platform for an Israeli client. Speakers upload lectures, users watch and listen, create playlists, and donate to campaigns.

I was the lead developer from our side. I assigned tickets to developers, reviewed and merged pull requests, and deployed releases to production.

I also built features myself — a like system, editable playlists, legacy speakers pages, and a most-viewed filter. On the backend I optimised REST APIs and background processing so the platform stayed fast under high traffic.

I worked with Docker for containers, Redis for caching, MySQL as the main database, and queues for background jobs. This project grew my DevOps skills a lot."

How to explain the features technically (*say only what matches what you really built*) - **Like button:** "A likes table with a unique key on user + lecture so a user can like only once. The API toggles like/unlike. The like count is stored on the lecture and updated in the same transaction, so we don't count rows on every page load." - **Editable playlists:** "Playlist and playlist-items tables with a position column. Reordering updates positions in one transaction. Only the owner can edit — checked in middleware." - **Most-viewed filter:** "View counts are recorded in the background, not in the request path. The top lists are cached in Redis with a TTL, so the database isn't hit on every request." - **Background processing:** "Heavy work like view-count updates, notifications and media processing goes to a queue (RabbitMQ) and is processed by workers, so the API responds quickly."

Deep-dive Q&A - What did your PR review check? "Correct logic, edge cases, SQL performance (indexes, N+1 queries), security like input validation and auth checks, naming, and that tests or manual test steps exist." - **How did you deploy?** "Code merged to the main branch, Docker image built, deployed to the server, then smoke-tested key pages. If something broke, we rolled back to the previous image." - **How did you handle high traffic?** "Caching hot data in Redis, indexes on frequent filters, pagination, moving heavy work to background workers, and horizontal scaling of containers." - **How did you work with the client?** "Calls and Slack/Jira with the client team in Israel, understanding tickets, giving estimates, and demoing features."

PROJECT 4 — BikeShopShippers (Morpheme Webnexus) ★ resume project

Stack: Medusa.js (Node.js commerce framework), PostgreSQL, Redis, Stripe API, UPS API, Google Places API, Next.js/React, GitHub Actions CI/CD

What it is: A logistics platform for shipping heavy items like bikes and vehicles. The platform makes bulk deals with shipping companies for discounts. Shop owners register and ship through it at lower rates.

Spoken explanation

"BikeShopShippers is a logistics platform. Bike and vehicle shop owners need to ship heavy items to customers, which is expensive. The platform negotiates bulk discounts with carriers, and shop owners book shipments through it.

I built it from scratch on Medusa.js, which is a Node.js headless commerce framework, with PostgreSQL. I integrated UPS APIs for rates, packaging, shipping labels and tracking, Stripe for payments, and Google APIs for address autocomplete. I also built the shipment dashboard UI.

I set up CI/CD with GitHub Actions and used GitHub Projects to plan and assign tasks, and I led the releases.

One challenge was keeping payment and shipment in sync. I used Stripe webhooks — the label is created only after Stripe confirms payment, and webhook handling is idempotent so a retry doesn't create two labels."

Deep-dive Q&A - Why Medusa.js? "It gives ready commerce modules — orders, customers, payments, fulfilment providers — and it's extendable in Node. I added a custom UPS fulfilment provider instead of building commerce from zero." - **How does Stripe payment work?** "Backend creates a PaymentIntent, frontend confirms the card with Stripe.js, and Stripe sends a webhook. I verify the webhook signature and then mark the order paid." - **UPS integration?** "OAuth token from UPS, then Rating API for prices, Shipping API for labels, and Tracking API for status updates." - **What did the CI/CD pipeline do?** "On push: install, lint, test and build. On merge to main: deploy to the server automatically."

PROJECT 5 — ViewReports BI platform (Atiya) — great backend story

Stack: Express 5, SQL Server, PostgreSQL, SQLite read-model, Redis, BullMQ, node-cron, JWT + email OTP, ExcelJS, React

"Management needed 45+ call-centre reports — agent performance, campaign conversion, transfers, revenue. Source tables had 50 million+ rows, so live queries were too slow.

I built an ETL sync that copies data in chunks using keyset pagination into a local SQLite read-model. A BullMQ worker runs the sync on a schedule. Reports read from the read-model with caching that is invalidated when new data syncs.

Login uses JWT plus an email OTP, and every report has a permission key for RBAC. Excel export has a row-limit guard (around 800k rows) that returns HTTP 413 instead of crashing the server.

Challenge: one heavy report on synchronous SQLite blocked the Node event loop and froze the app. I added a date-range guard, moved index building to a worker thread and separated the sync worker. After that there were no freezes."

PROJECT 6 — QMS call-quality report (Atiya) — query optimisation story

"QA auditors needed all details of one call — both call legs, agent, disposition and the recording. The original SQL Server view used a per-row OUTER APPLY and took about 92 seconds.

I bypassed the view, queried base tables with UNION ALL, and added the right indexes. It came down to 200–350 milliseconds. I also built an audio streaming proxy that supports HTTP Range requests so recordings play and seek inside the page."

PROJECT 7 — CallDropAutoDial (Atiya) — automation story

"When customer calls dropped or were abandoned, nobody called back. I built a Node service with node-cron that runs every 30 minutes, finds dropped calls in PostgreSQL using DISTINCT ON and NOT EXISTS, and uploads them to dialer campaigns for automatic call-back.

Safety: a DRY_RUN flag to test without dialling, a re-entrancy guard so two runs don't overlap, and only one process on IIS so the cron never fires twice.

Interesting bug: results were always empty. The table I read was filled by an hourly ETL job, so the 30-minute window was always empty. I switched to the live parent table."

PROJECT 8 — Patient-CRM (Atiya) — caching story

"Agents search a phone number and see the patient's full 30-day call history across all their numbers. The first version took minutes. I added a 3-tier cache: in-memory LRU, then Redis, then a SQLite mirror, and finally PostgreSQL. Lookups dropped to milliseconds. If Redis is down, the code falls through to the next tier — slower, but not broken."

Other projects (short answers if asked)

Project	Company	2-line answer
PERK	Morpheme	Event management platform. I built serverless CRUD APIs on AWS Lambda with TypeScript, a PDF-to-image converter and a PDF editor for event layout maps.
FamilyOne	Morpheme	Family discount network. I optimised Lambda functions, wrote serverless YAML configs, load-tested APIs with JMeter and set up Sentry and Codecov.
Nosotros LMS	Morpheme	LMS for Spanish schools with admin, school, teacher, parent and student portals. I set up Bitbucket CI/CD with Sentry, Codecov, Jira and Slack.
ShowTrail	Morpheme	Coupon promotion platform sharing FamilyOne's admin. I trained Intercom AI workflows for customer support.
SmartBiz	Morpheme	B2B onboarding with dynamic forms per sector. I built the APIs and UI.

Project	Company	2-line answer
Astrology platform	MSAI	Built from scratch: tech stack, Prokerala astrology APIs, chat + audio/video consultation.
ThePNAC.com	MSAI	Built end to end and deployed with PM2 + NGINX on the company server.
Real Estate	Personal	MERN + Firebase: listings, filters, buyer/seller dashboards. Deployed on Render.
E-commerce	Personal	MERN: auth, products, cart, orders, payments, admin dashboard.

Common project questions (ready answers)

"What was the most challenging project?" → ViewReports (scale + event loop freeze) or QMS (92 s → 300 ms).

"Did you build it alone?"

"At Atiya, yes — I'm the main developer. I worked with business users for requirements and testing, and the IT team for servers and database access. At Morpheme I worked in a team and led TorahAnytime."

"What would you improve in your project?"

"Add TypeScript, more unit tests per service, move secrets to a vault, add a CI/CD pipeline for the IIS apps, and add the Redis adapter for Socket.IO scaling."

"How many users?" → Give honest approximate numbers: "Used daily by the call-centre agents and managers" (confirm the real count — e.g., 100+ agents).

05 — Education and Personal Background

1. Education summary

Level	Institution	Board / University	Year	Score
B.Tech, Computer Science Engineering	Technocrats Institute of Technology (TIT), Bhopal, MP	RGPV (Rajiv Gandhi Proudhyogiki Vishwavidyalaya)	2019 – 2023	CGPA 8.7 / 10
12th (Senior Secondary)	Govt. Inter College, Lalitpur, UP	UP Board	2019	76%
10th (Higher Secondary / High School)	SDRS Inter College, Lalitpur, UP	UP Board	2017	76%

Check these against your marksheets. Older intros said CGPA 8.5 — your resume says **8.7**. Always say **8.7** to match the resume.

2. "Tell me about your education" (spoken answer)

"I did my schooling in Lalitpur, Uttar Pradesh, and scored 76% in both 10th and 12th. Then I completed my B.Tech in Computer Science Engineering from Technocrats Institute of Technology, Bhopal, affiliated to RGPV, from 2019 to 2023, with 8.7 CGPA. In college I learned C, C++, Java, data structures, DBMS, operating systems and computer networks. I built a packers-and-movers management system in Java and a dictionary application in C++. In my final year I started web development — HTML, CSS, JavaScript and then the MERN stack — and joined Shaligram Infotech in February 2023 as a software developer."

3. Education follow-up questions

Why did you choose Computer Science?

"I was interested in computers from school. I liked the logic of programming and how software solves real problems. CS also gave me a good career for supporting my family."

Why did you choose TIT Bhopal?

"It has a good CS department and placement record, it's affiliated with RGPV, and it was closer to my home in Ashok Nagar."

What was your favourite subject?

"DBMS. I liked how data is structured, normalised and queried. It helps me a lot today — query optimisation is one of my strongest skills." *(Be ready for: normalisation, primary/foreign keys, joins, indexes, ACID — see file 06.)*

What was your final-year project?

"A packers-and-movers management system in Java with a database — customers could request a move, and the admin could manage bookings and status." *(Adjust if your final-year project was different.)*

If you could do college again, what would you change?

"I would start web development and DSA practice earlier, and take part in more hackathons and tech events."

Any certifications?

Mention any real ones (online courses on Node.js, AWS, MongoDB). If none: "I mostly learned by building real projects at work, like BullMQ queues, Redis caching and AWS Lambda. I'm planning the AWS Developer Associate certification."

Did you have any gap?

"No. I completed B.Tech in 2023 and joined Shaligram Infotech in February 2023 during my final semester." *(Make sure this matches your documents.)*

Why was your 12th completed in 2019 and 10th in 2017? *(only if asked)*

Give the true reason briefly (for example a change of school or one extra year). Keep it short and positive.

4. Personal background

Where are you from?

"I'm from Ashok Nagar district in Madhya Pradesh. My native village is Hirawal. It's a small and peaceful village where most people are farmers. Currently I live in Delhi."

Tell me about your family.

"We are a family of five. My father is a farmer and my mother is a homemaker. I have a younger sister who is studying, and a younger brother who is also in school. My parents always supported my education even with limited resources, and that motivates me to work hard."

Tell me about your childhood.

"I grew up in a village. I saw my father work hard on the farm every day, which taught me discipline and patience. I studied in Lalitpur, played a lot of cricket, and became interested in computers in school."

What are your hobbies?

"Playing cricket, guiding students who are starting programming, and exploring new tech tools. I also like travelling to natural places."

Who is your role model?

"My father — for his hard work and honesty."

Languages you speak?

"Hindi and English."

5. Small tips

- In a **technical round**, keep education to 2–3 lines. Focus on experience.
- In an **HR round**, family and hometown answers should be short, warm and positive (30 seconds).
- Don't mention your age unless asked.
- Never sound sad about a village or farming background — it shows your hard work and values.

06 — Role, Tech Stack and Node.js Backend Q&A

How to answer technical questions: 1) Simple definition → 2) How it works (1–2 lines) → 3) Where I used it in my project. Step 3 is what makes you sound experienced.

PART 1 — Explaining your role and tech stack

"What is your role in your current company?"

"I'm a MERN stack developer with a backend focus. I own the backend systems end to end: I take requirements from business users, design APIs and database queries, write the Node.js code, integrate third-party APIs, deploy to IIS or Cloudways, and support the apps in production. I also build the React screens when needed."

"Explain your tech stack."

"My core stack is Node.js with Express for the backend. For databases I use MongoDB with Mongoose, and SQL databases — SQL Server, PostgreSQL and MySQL. I use Redis for caching and as a queue broker with BullMQ. For real-time I use Socket.IO. On the frontend I use React with Redux Toolkit and Tailwind. For deployment I've used Docker, PM2, NGINX, IIS with iisnode, AWS Lambda, EC2 and S3, with CI/CD on GitHub Actions and Bitbucket."

"What does your typical day look like?"

"Morning: check production logs and any issues from users. Then requirement discussion if a new feature is coming. Then development — usually backend first: SQL query, repository, service, controller, route and validation, tested in Postman. Then the frontend screen. Then build, deploy, check health endpoints and verify with real users. I commit with clear messages and update documentation."

"How do you structure a Node.js project?"

"Layered structure: routes (only HTTP mapping) → validations (Joi schemas) → controllers (thin, handle request/response) → services (business logic) → repositories (database queries). Plus middlewares (auth, error handler, rate limiter), config (env validation), utils (ApiError, ApiResponse, logger). Each feature becomes 5 small files, easy to test and easy for others to find."

```
src/  
  config/      env.js, db.js, redis.js  
  routes/      appointment.routes.js  
  validations/ appointment.validation.js  
  controllers/ appointment.controller.js  
  services/     appointment.service.js  
  repositories/ appointment.repository.js  
  middlewares/  auth.js, errorHandler.js, rateLimit.js  
  utils/        ApiError.js, ApiResponse.js, logger.js  
  app.js, server.js
```

PART 2 — JavaScript core (asked in almost every Node interview)

1. var vs let vs const? `var` is function-scoped and hoisted with `undefined`. `let` and `const` are block-scoped and in the "temporal dead zone" until declared. `const` cannot be reassigned, but object contents can change.

2. What is hoisting? Declarations move to the top of their scope before execution. Function declarations are fully hoisted; `var` is hoisted as `undefined`; `let/const` are hoisted but not initialised.

3. What is a closure? A function that remembers variables from its outer scope even after the outer function has finished.

```
function counter() { let c = 0; return () => ++c; }
const next = counter(); next(); // 1
next(); // 2
```

Used for: private variables, memoisation, middleware factories like `authorize(role)`.

4. == vs ===? `==` converts types before comparing (`'5' == 5` is true). `===` compares value and type. Always use `===`.

5. Promise, async/await? A Promise represents a future value: pending → fulfilled or rejected. `async/await` is cleaner syntax over promises. Errors are handled with `try/catch`.

6. Promise.all vs allSettled vs race vs any? - `all`: waits for all; fails fast if one rejects. - `allSettled`: waits for all, gives status of each — I use it when one failed API should not fail the whole report. - `race`: first to settle (useful for timeouts). - `any`: first to fulfil.

7. Event loop, microtask vs macrotask? Call stack runs sync code. Then all **microtasks** run (promise callbacks, `process.nextTick` first in Node). Then one **macrotask** (`setTimeout`, I/O). Output of:

```
console.log(1); setTimeout(()=>console.log(2)); Promise.resolve().then(()=>console.log(3));
console.log(4);
// 1 4 3 2
```

8. this keyword and arrow functions? `this` depends on how a function is called. Arrow functions don't have their own `this`; they take it from the outer scope.

9. call, apply, bind? All set `this`. `call(obj, a, b)` runs now with arguments; `apply(obj, [a, b])` runs now with an array; `bind(obj)` returns a new function for later.

10. Shallow vs deep copy? Spread `{...obj}` copies one level. Deep copy: `structuredClone(obj)`.

11. Debounce vs throttle? Debounce runs after the user stops for X ms (search box). Throttle runs at most once every X ms (scroll).

12. map vs forEach vs filter vs reduce? `map` returns a new transformed array; `forEach` returns nothing; `filter` returns matching items; `reduce` builds one value (sum, grouped object).

13. What is prototype inheritance? Every object has a hidden link to a prototype object. Property lookup goes up this chain. `class` syntax is built on prototypes.

14. Spread vs rest? Same `...` syntax. Spread expands (`[...arr]`), rest collects (`function f(...args)`).

PART 3 — Node.js

15. What is Node.js? A JavaScript runtime built on Chrome's V8 engine. It's single-threaded with a non-blocking, event-driven I/O model, which makes it great for I/O-heavy apps like APIs and real-time systems.

16. How does Node handle many requests with one thread?

"The main thread runs JavaScript. I/O like database calls, file reads and network calls is handed to the OS or libuv's thread pool. When the work finishes, a callback is queued and the event loop runs it. So the thread is never waiting idle. But CPU-heavy code blocks everything." *My example:* "In my reporting app, a heavy synchronous SQLite query blocked the event loop and froze the app. I added a date-range guard and moved index building to a worker thread."

17. Event loop phases? Timers (setTimeout/setInterval) → pending callbacks → poll (I/O) → check (setImmediate) → close callbacks. `process.nextTick` and promise microtasks run between phases.

18. setImmediate vs setTimeout(0) vs process.nextTick? `nextTick` runs right after the current operation, before other microtasks. `setImmediate` runs in the check phase after I/O. `setTimeout(0)` runs in the timers phase. Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)`.

19. What is libuv? The C library behind Node's event loop and thread pool (default 4 threads) used for file system, DNS, crypto and compression.

20. Worker threads vs cluster vs child process? - **Worker threads:** run CPU-heavy JS in parallel inside one process (shared memory possible). - **Cluster:** runs multiple Node processes on the same port to use all CPU cores. PM2 cluster mode does this. - **Child process:** run another program or script (`spawn` , `exec` , `fork`).

21. What are streams? Types? Process data piece by piece instead of loading everything into memory. Types: Readable, Writable, Duplex, Transform. Use `pipeline()` for safe piping with error handling.

My example: "I stream call recordings from the dialer to the browser with HTTP Range support, and I use database cursors to sync large tables in batches."

22. What is a Buffer? Raw binary data in memory outside the V8 heap, used for files, images and network data.

23. CommonJS vs ES Modules? CommonJS: `require` / `module.exports` , loaded synchronously. ESM: `import` / `export` , static and async, needs `"type": "module"` . In ESM there's no `__dirname` — use `import.meta.url` . My Atiya backends use ESM.

24. What is middleware? (Node/Express) A function (`req` , `res` , `next`) that runs between the request and the response — for logging, auth, validation, parsing or error handling. Calling `next()` passes control forward.

25. How do you handle errors in Node?

" try/catch with async/await, a central Express error-handler middleware, a custom `ApiError` class with status codes, and process-level handlers for `unhandledRejection` and `uncaughtException` that log the error. For real bugs the process exits and PM2 or IIS restarts it."

26. How do you do graceful shutdown? On `SIGTERM` / `SIGINT` : stop accepting new connections (`server.close()`), finish running requests, close DB and Redis connections, then exit. Add a 10-second

forced-exit timer.

27. How do you find a memory leak? Watch memory in PM2 or monitoring. Take heap snapshots with `--inspect` and Chrome DevTools and compare. Common causes: global arrays or caches that keep growing, event listeners not removed, unclosed timers. Fix with bounded LRU caches and cleanup.

28. How do you improve Node.js performance?

"Use indexes and optimised queries, cache hot data in Redis, paginate, stream large data, avoid blocking the event loop, use cluster mode or more instances, compression, connection pooling, and move heavy work to background queues."

29. What is package-lock.json? It locks exact dependency versions so every install is the same. Use `npm ci` in CI/CD.

30. dependencies vs devDependencies? `dependencies` are needed at runtime (express). `devDependencies` only for development (jest, eslint). In production: `npm ci --omit=dev`.

31. How do you manage environment config? `.env` files per environment, loaded with dotenv, validated at startup with a schema (envalid / Joi) so the app fails fast if a variable is missing. Never commit real secrets.

PART 4 — Express.js

32. What is Express? A minimal web framework for Node that provides routing, middleware and helpers for request/response.

33. Middleware order in your apps? `helmet` → `cors` → `compression` → body parser with size limit → request ID → logger → rate limiter → auth → routes → 404 handler → error handler. Webhook routes are mounted **before** auth because the provider calls them without a token.

34. Error-handling middleware? Has 4 arguments (`err`, `req`, `res`, `next`) and must be registered last.

```
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  logger.error(err);
  res.status(status).json({ success: false, message: status === 500 ? 'Internal error' :
err.message });
});
```

35. Express 4 vs Express 5? Express 5 automatically catches rejected promises in async handlers (no `express-async-errors` needed), `req.query` is read-only, and route path syntax changed.

36. app.use vs app.get? `app.use` mounts middleware for all methods on a path prefix. `app.get` handles only GET on an exact route.

37. req.params vs req.query vs req.body? `params`: from URL path `/users/:id`. `query`: from `?page=2`. `body`: from POST/PUT payload.

38. How do you validate input? Joi schemas through a generic `validate(schema)` middleware for body, query and params. Invalid input returns 400 with clear messages.

39. How do you upload files? `multer` with size limits and file-type checks. Save to a temp folder or memory, then upload to S3 or Cloudinary. (*I used multer + Cloudinary in the WhatsApp platform.*)

40. How do you implement CORS? `cors` middleware with an allowlist of origins. Or avoid CORS by serving the API on the same domain through a reverse proxy.

PART 5 — REST API design

41. What is REST? An architectural style where resources are identified by URLs and manipulated with HTTP methods. It's stateless — every request carries everything needed.

42. HTTP methods? GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove). GET, PUT, DELETE are idempotent; POST is not.

43. Important status codes? 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (no permission), 404 Not Found, 409 Conflict, 413 Payload Too Large, 422 Validation error, 429 Too Many Requests, 500 Server Error, 502 Bad Gateway, 503 Service Unavailable.

44. PUT vs PATCH? PUT replaces the whole resource. PATCH updates only the given fields.

45. How do you design pagination? Offset pagination (`?page=2&limit=20`) is simple but slow for big tables. **Keyset/cursor pagination** (`WHERE id > lastId ORDER BY id LIMIT 20`) is fast for large data.

My example: "I used keyset pagination to sync tables with 50M+ rows."

46. How do you version APIs? In the URL (`/api/v1/...`) — simplest and most common. Old versions stay until clients move.

47. What is idempotency and why does it matter? Doing the same request many times gives the same result. Important for payments and webhooks.

My example: "Exotel can resend a webhook. I upsert by the message ID, so duplicates don't create two records. For Stripe I use idempotency keys."

48. REST vs GraphQL vs tRPC? REST: multiple endpoints, simple caching. GraphQL: one endpoint, client chooses fields, avoids over-fetching. tRPC: type-safe calls between a TypeScript frontend and backend without a schema — used in TorahAnytime.

49. How do you document APIs? Swagger / OpenAPI served at `/api/docs` (used in the Hospital CRM), plus Postman collections.

50. How do you call a third-party API safely? Timeouts, retry with exponential backoff only for network errors / 5xx / 429 (never 4xx), circuit breaker for repeated failures, logging, and secrets from env. Never let a third-party failure crash the main flow.

PART 6 — Authentication and security

51. Authentication vs authorization? Authentication: who are you (login). Authorization: what are you allowed to do (roles, permissions).

52. How does JWT work? After login the server signs a token with a secret. It has 3 parts: header, payload, signature. The client sends it in `Authorization: Bearer <token>`. The server verifies the signature — no session lookup needed.

53. Access token + refresh token? Short-lived access token (15 min) for API calls, long-lived refresh token (7 days) to get a new access token. Store refresh token in an httpOnly secure cookie and in the DB so it can be revoked.

54. Where to store JWT on the frontend? httpOnly, Secure, SameSite cookie is safest against XSS. localStorage is simpler but vulnerable to XSS.

55. How do you store passwords? Hash with bcrypt (salt + cost factor ~10–12). Never store plain text or use reversible encryption for new systems.

56. How did you implement RBAC?

"Users have roles and permission keys. A middleware like `authorize('report:view')` checks the permission on each route. The same permission keys hide menu items in the frontend. In ViewReports every report has its own permission key."

57. How did you implement OTP two-factor login?

"After password check, I generate a 6-digit OTP, store it in Redis with a 5-minute TTL and an attempt counter, and email it. The user submits it, I compare it in constant time, then issue the JWT."

58. What is OAuth 2.0? A standard for delegated access. The user logs in with Google, Google gives an authorization code, the backend exchanges it for tokens. Used for "Login with Google".

59. Common security practices in Node APIs? - Helmet for security headers - CORS allowlist - Rate limiting (express-rate-limit) - Input validation (Joi / zod) - Parameterised queries (prevent SQL injection) - Sanitise MongoDB queries (prevent NoSQL injection like `{ "$gt": "" }`) - Secrets in env, not in code - HTTPS everywhere - `npm audit` / Dependabot for vulnerable packages

60. SQL injection — how to prevent? Never build SQL by joining strings with user input. Use parameterised queries: `request.input('id', sql.Int, id)` in mssql or `$1` in pg.

61. XSS and CSRF? XSS: attacker injects script into pages — escape output, use CSP, httpOnly cookies. CSRF: attacker makes the browser send a request with your cookie — use SameSite cookies or CSRF tokens.

62. How do you secure webhooks? Verify the provider's signature (HMAC) or a secret token, respond 200 quickly, process idempotently, and store the raw payload for audit.

PART 7 — MongoDB

63. SQL vs NoSQL — when to use which? SQL: structured data, relations, transactions (orders, appointments, payments). NoSQL (MongoDB): flexible schemas, documents, high write rate (chat messages, logs, scraped data).

My example: "Hospital CRM uses SQL Server because data is relational. The WhatsApp platform uses MongoDB because messages have varied payloads."

64. What is an index? Types in MongoDB? A data structure that makes queries faster (like a book index) but slows writes slightly. Types: single field, compound, multikey (arrays), text, TTL, unique, partial.

65. Compound index and the ESR rule? Order fields as **E**quality → **S**ort → **R**ange. Example: `{ to: 1, isRead: 1, timestamp: -1 }`.

66. How do you check if a query uses an index? `.explain('executionStats')` — look for `IXSCAN` (good) vs `COLLSCAN` (full scan), and compare docs examined vs returned.

67. Aggregation pipeline? A sequence of stages that transform data: `$match` → `$group` → `$sort` → `$project` → `$lookup` (join). Put `$match` first to use indexes.

```
Message.aggregate([
  { $match: { to: myNumber, isRead: false } },
  { $group: { _id: '$from', unread: { $sum: 1 } } },
  { $sort: { unread: -1 } }
]);
```

68. Embedding vs referencing? Embed when data is read together and small (address inside user). Reference when data is large, shared or grows without limit (messages of a contact).

69. Mongoose populate vs \$lookup? `populate` does a second query from the app. `$lookup` joins inside the database in one aggregation.

70. Transactions in MongoDB? Supported on replica sets with `session.startTransaction()`. Use them when several documents must change together.

71. What are Mongoose middleware (hooks)? `pre('save')` and `post('save')` functions — for example, hashing a password before save.

72. Sharding vs replication? Replication: copies of the same data for high availability (replica set). Sharding: splits data across servers for horizontal scale.

PART 8 — SQL (MySQL / PostgreSQL / SQL Server)

73. Types of joins? INNER (matching rows), LEFT (all from left + matches), RIGHT, FULL OUTER, CROSS, SELF join.

74. WHERE vs HAVING? WHERE filters rows before grouping. HAVING filters groups after GROUP BY.

75. Clustered vs non-clustered index? Clustered: table rows are physically stored in index order (one per table, often the primary key). Non-clustered: separate structure that points to rows (many per table).

76. What is normalisation? Organising tables to remove duplicate data: 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency). Reporting tables are sometimes denormalised for speed.

77. ACID? Atomicity (all or nothing), Consistency (valid state), Isolation (transactions don't interfere), Durability (saved after commit).

78. How did you optimise a slow query? (92 s → 300 ms)

"I read the execution plan. The SQL Server view used OUTER APPLY, which ran a sub-query for every row. I bypassed the view, queried base tables directly with UNION ALL for the two call legs, and added indexes on the filter and join columns. Time dropped from about 92 seconds to 200–350 ms."

79. Query optimisation checklist? Read the execution plan → add indexes on WHERE/JOIN/ORDER BY columns → select only needed columns (no `SELECT *`) → avoid functions on indexed columns in WHERE → avoid N+1 queries → paginate → use proper data types → cache results.

80. What is the N+1 problem? Fetching a list (1 query) and then one query per item (N queries). Fix with a JOIN, `IN (...)`, `$lookup` or ORM eager loading.

81. Stored procedure vs function vs view? Procedure: saved SQL logic, can modify data. Function: returns a value or table, used inside queries. View: saved SELECT used like a table.

82. What is a deadlock? Two transactions each wait for a lock the other holds. Prevent by accessing tables in the same order, keeping transactions short and using the right indexes.

83. Second highest salary?

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);  
-- or  
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employees) t  
WHERE r = 2;
```

84. Find duplicate records?

```
SELECT phone, COUNT(*) FROM patients GROUP BY phone HAVING COUNT(*) > 1;
```

85. PostgreSQL features you've used? `DISTINCT ON` (first row per group), `NOT EXISTS` for anti-joins, partial indexes, server-side cursors (`DECLARE ... FETCH 10000`) for big syncs.

86. Connection pooling — why? Opening a DB connection is slow. A pool keeps connections open and reuses them. Set a max size so the DB isn't overloaded.

PART 9 — Redis, caching and queues

87. What is Redis and where did you use it? An in-memory key-value store — very fast. I used it for caching reports and lookups, OTP storage with TTL, session state, rate limiting, and as the broker for BullMQ queues.

88. Caching strategies? - Cache-aside (most used): read cache → if miss, read DB → store in cache with TTL. - **Write-through:** write cache and DB together. - **Invalidation:** delete or version the key when data changes.

My example: "Cache keys include a version and the data-sync timestamp, so after a new sync, old cache is automatically ignored. I only cache successful, non-empty results."

89. What happens if Redis goes down?

"Every cache call is wrapped in try/catch. The app logs it and falls back to the database. Slower, but not broken."

90. Redis data types? String, Hash, List, Set, Sorted Set (leaderboards, top-viewed), Stream, plus TTL on any key.

91. Why use a message queue? To move slow or unreliable work out of the request: emails, notifications, reports, scraping. Gives retries, scaling of workers and smoothing of traffic spikes.

92. BullMQ — what do you know? Redis-based job queue for Node. Queue adds jobs; Worker processes them. Options: `attempts`, `backoff` (exponential), `concurrency`, repeatable (cron) jobs, `removeOnComplete`. I

used it for a 4-stage scraping pipeline and the ViewReports sync worker.

93. RabbitMQ vs Kafka vs BullMQ? - **BullMQ**: simple job queue on Redis, great for Node background jobs. - **RabbitMQ**: message broker with exchanges and routing; good for task distribution between services (used in TorahAnytime). - **Kafka**: distributed log for very high-throughput event streaming and replay.

94. node-cron vs a queue? node-cron is fine for simple scheduled jobs in a single process. A queue is better when you need retries, multiple workers or persistence. With multiple instances, cron can run twice — use one instance or a distributed lock.

PART 10 — AWS, serverless and cloud

95. AWS services you've used? Lambda (serverless functions), S3 (file storage), EC2 (servers), RDS (managed SQL), API Gateway (HTTP in front of Lambda), CloudWatch (logs).

96. What is serverless? Pros and cons? You write functions; AWS runs and scales them, and you pay per request. Pros: no server management, auto-scaling, cost-efficient at low traffic. Cons: cold starts, execution time limits (15 min), harder local testing, vendor lock-in.

97. What is a cold start and how do you reduce it? The first request after idle needs a new container, which adds latency. Reduce by smaller bundles, fewer dependencies, initialising DB connections outside the handler (reused), more memory, or provisioned concurrency.

My example: "In FamilyOne I optimised Lambda functions and serverless YAML configs and load tested them with JMeter."

98. How do you upload files to S3 securely? Generate a **pre-signed URL** on the backend. The client uploads directly to S3. The backend never handles the big file, and the bucket stays private.

99. EC2 vs Lambda? EC2: full server, always on, good for long-running apps and WebSockets. Lambda: event-driven, short functions, scales to zero.

PART 11 — Docker, deployment and CI/CD

100. What is Docker? Why use it? Packages an app with its dependencies into an image that runs the same everywhere. Solves "works on my machine".

101. Image vs container? An image is the template (read-only). A container is a running instance of an image.

102. Sample Dockerfile for Node:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Copying `package.json` first uses layer caching, so dependencies reinstall only when they change.

103. Docker Compose? Runs multiple containers together (API + MongoDB + Redis) with one `docker-compose up`.

104. Kubernetes basics? Orchestrates containers across servers. Pod (one or more containers), Deployment (desired replicas + rolling updates), Service (stable network address / load balancing), Ingress (HTTP routing), HPA (auto-scaling).

105. What is PM2? A process manager for Node: keeps the app alive, restarts on crash, cluster mode for multiple cores, log management, `max_memory_restart`.

106. What is NGINX used for? Reverse proxy in front of Node, SSL termination, load balancing, serving static files, gzip and WebSocket proxying.

107. How did you deploy on IIS?

"IIS with iisnode. I wrote `web.config` with the iisnode handler, URL rewrite rules, number of Node processes and logging. App pool set to No Managed Code and AlwaysRunning. The React build is served as a static site with an SPA rewrite to `index.html`. One gotcha: on iisnode, `process.env.PORT` is a named pipe, not a number — so never `parseInt` it."

108. What is CI/CD? CI: every push runs install, lint, tests and build automatically. CD: after merge, the app is deployed automatically. I set it up with GitHub Actions (BikeShopShippers) and Bitbucket Pipelines (Nosotros), with Sentry and Codecov.

109. Sample GitHub Actions flow? On push → checkout → setup Node → `npm ci` → lint → test → build → (on main) deploy via SSH or Docker image push.

110. Zero-downtime deployment? PM2 `reload` (restarts cluster workers one by one), rolling updates in Kubernetes, or blue-green deployment behind a load balancer.

111. How do you monitor production? Structured logs (Winston), error tracking (Sentry), health endpoints (`/health`, `/health/ready`), uptime checks and alerts.

PART 12 — System design and microservices

112. Monolith vs microservices? Monolith: one codebase and deployment — simple, good to start. Microservices: small independent services with their own data — scale and deploy separately, but more complex (network calls, monitoring, data consistency). Start monolith, split when needed.

113. How do microservices communicate? Synchronous: REST, gRPC. Asynchronous: message queues (RabbitMQ, Kafka) and events. An API gateway is the single entry point for clients.

114. How do you scale a Node.js API? Vertical (bigger server) → Horizontal (more instances behind a load balancer, PM2 cluster) → keep the app stateless (sessions in Redis/JWT) → cache → DB read replicas and indexes → queues for heavy work → CDN for static files.

115. What is a load balancer? Distributes traffic across servers (round robin, least connections). Also does health checks.

116. Rate limiter design? Count requests per user/IP in Redis with a time window (fixed window, sliding window, token bucket). Return 429 when exceeded.

117. Design a URL shortener (common question)? API: POST `/shorten` returns a short code; GET `/:code` redirects. Code: base62 of an auto-increment ID or random 7 chars. Store code → URL in DB with a unique index. Cache hot codes in Redis. Use 301/302 redirects. Track clicks asynchronously through a queue.

118. Design a notification / WhatsApp system (use your project)?

"Producer API saves the message as `queued` and pushes a job to a queue. Workers send it to the provider with retries and backoff. Provider webhooks update status by message ID (idempotent). Socket.IO pushes updates to users. Rate limits per provider. Store raw payloads for audit."

119. CAP theorem? In a distributed system with a network partition you choose Consistency or Availability. MongoDB replica sets favour consistency by default; many caches favour availability.

120. How do you handle distributed transactions? Saga pattern: a series of local transactions, each with a compensating action if a later step fails. Or the outbox pattern for reliable event publishing.

PART 13 — Real-time: WebSockets, Socket.IO, WebRTC

121. HTTP polling vs WebSocket vs SSE? Polling: client asks repeatedly (simple, wasteful). WebSocket: full two-way persistent connection (chat). SSE: server-to-client one-way stream over HTTP (progress, notifications).

122. Socket.IO features? Auto-reconnect, rooms, namespaces, fallback to polling, acknowledgements. Scale with the Redis adapter + sticky sessions.

123. What is WebRTC? (*prepare if WebRTC stays on your resume*) Peer-to-peer audio, video and data in the browser. - **Signaling**: peers exchange SDP offer/answer and ICE candidates through your server (usually WebSocket/Socket.IO). - **STUN**: tells a peer its public IP. - **TURN**: relays media when direct connection fails (strict NAT/firewall). - Media is encrypted (DTLS-SRTP). - For many participants, use an SFU (media server) or a provider like ZegoCloud/Twilio/Agora.

Safe answer: "For doctor–patient consultation, the backend creates a session for the appointment, authenticates both users, and handles signaling over WebSocket. Media flows peer-to-peer, with a TURN server as fallback."

PART 14 — Testing, TypeScript, Nest.js, Prisma

124. How do you test APIs? Unit tests for services with Jest (mock repositories), integration tests for endpoints with Supertest, Postman collections for manual testing, load tests with JMeter.

```
test('GET /health returns 200', async () => {
  const res = await request(app).get('/health');
  expect(res.statusCode).toBe(200);
});
```

125. Unit vs integration vs e2e? Unit: one function in isolation. Integration: several parts together (API + DB). E2E: the whole system like a real user.

126. Why TypeScript? Static types catch errors at compile time, give better autocomplete and make refactoring safer. I used it in AWS Lambda projects at Morpheme.

127. interface vs type in TypeScript? Both describe shapes. `interface` can be extended and merged; `type` can also describe unions and primitives.

128. What is Nest.js? A TypeScript Node framework with an Angular-like structure: modules, controllers, providers (services), dependency injection, guards (auth), pipes (validation), interceptors. Good for large structured backends.

129. What is Prisma? A type-safe ORM. Define models in `schema.prisma`, run migrations, and use the generated client: `prisma.user.findMany({ where: { active: true } })`.

130. Sequelize vs Prisma vs raw SQL? Sequelize: older, model-based ORM. Prisma: modern, type-safe. Raw SQL: full control and best for complex reports — which is why I used it for legacy SQL Server.

PART 15 — Common live coding questions (practise these)

```
// 1. Reverse a string
const reverse = s => s.split('').reverse().join('');

// 2. Remove duplicates from an array
const unique = arr => [...new Set(arr)];

// 3. Flatten a nested array
const flat = arr => arr.flat(Infinity);

// 4. Count character frequency
const freq = s => [...s].reduce((m, c) => (m[c] = (m[c] || 0) + 1, m), {});

// 5. Debounce
function debounce(fn, ms) {
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms); };
}

// 6. Promise with retry and exponential backoff
async function retry(fn, attempts = 3, base = 250) {
  for (let i = 1; i <= attempts; i++) {
    try { return await fn(); }
    catch (e) { if (i === attempts) throw e; await new Promise(r => setTimeout(r, base * 2 ** (i - 1))); }
  }
}

// 7. Simple Express CRUD route with validation
router.post('/users', validate(userSchema), async (req, res, next) => {
  try { const user = await userService.create(req.body); res.status(201).json(user); }
  catch (e) { next(e); }
});

// 8. Auth middleware
const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ message: 'Invalid token' }); }
};

// 9. Group array of objects by key
const groupBy = (arr, k) => arr.reduce((acc, x) => ((acc[x[k]] || []).push(x), acc), {});

// 10. Implement Promise.all
function promiseAll(ps) {
  return new Promise((resolve, reject) => {
    const out = []; let done = 0;
    if (!ps.length) return resolve(out);
    ps.forEach((p, i) => Promise.resolve(p).then(v => { out[i] = v; if (++done === ps.length)
    resolve(out); }, reject));
  });
}
```

If you don't know an answer

"I haven't worked with that directly in production. The closest thing I've done is [X], where I [what you did]. I'd be happy to learn it quickly."

This sounds much better than guessing.

07 — Experience and Scenario Questions (STAR method)

The STAR method

Letter	Meaning	How long
S — Situation	Where, which project, what was the problem	1–2 lines
T — Task	What was your responsibility	1 line
A — Action	What YOU did, step by step (most important part)	3–5 lines
R — Result	Number or outcome, and what you learned	1–2 lines

Prepare these 10 stories well. One story can answer many different questions.

Story 1 — Slow query: 92 seconds to 300 ms

Use for: performance, problem solving, biggest technical achievement, debugging

- **S:** "At Atiya, QA auditors used a call-quality report. Loading one call took about 92 seconds, so they couldn't audit many calls."
 - **T:** "I had to make the report fast without changing the database used by other systems."
 - **A:** "I checked the execution plan. The SQL Server view used OUTER APPLY, which ran a sub-query for every row. I bypassed the view and queried the base tables directly, used UNION ALL for the two call legs, and added indexes on the filter and join columns. I also added query timeouts so a slow query couldn't hang the server."
 - **R:** "Load time dropped from about 92 seconds to 200–350 milliseconds. QA could audit many more calls per day. I learned to always read the execution plan before guessing."
-

Story 2 — App freezing in production (event loop blocked)

Use for: production issue, debugging under pressure, Node.js internals

- **S:** "In ViewReports, sometimes the whole app froze for all users, and IIS recycled the process."
 - **T:** "Find the cause and stop the freezes."
 - **A:** "From the logs, freezes matched big date-range reports. The SQLite driver is synchronous, so a heavy query blocked the Node event loop. I added a date-range guard on heavy reports, moved index building to a worker thread, moved the data sync to a separate BullMQ worker with concurrency 1, and cached results keyed by the sync timestamp."
 - **R:** "No more freezes, and reports load in milliseconds from cache. I learned never to do heavy synchronous work on the main thread."
-

Story 3 — Legacy migration with 460+ tables

Use for: biggest achievement, smart work, ownership

- **S:** "The old CRM was ASP.NET WebForms with 460+ tables in two SQL Server databases."
 - **T:** "Rebuild it in Node.js + React with the same behaviour, without changing the database."
 - **A:** "I read the legacy C# code to understand business rules and made a parity checklist. Instead of hand-writing models, I wrote a script that parsed the SQL schema and generated 463 model files, plus a script to scaffold controllers and routes. Then I built 47 API modules on a layered structure and tested each screen against the old system."
 - **R:** "The new CRM went live on IIS and agents use it daily inside the dialer, with no change in their workflow. Processing time improved by about 30%."
-

Story 4 — Data was always empty (wrong data source)

Use for: mistake you learned from, debugging, attention to detail

- **S:** "My CallDropAutoDial job ran every 30 minutes to find dropped calls, but it kept finding zero."
 - **T:** "Find why the list was empty."
 - **A:** "I ran the query manually with different time windows and checked when rows were inserted. The child table I used was filled by an ETL job only once an hour, so the last 30 minutes were always empty. I switched to the live parent table, tested with DRY_RUN mode first, and documented the reason."
 - **R:** "The job started finding the right calls and pushing them for call-back. I learned to always check how and when data is written before building on it."
-

Story 5 — Webhook race condition

Use for: tricky bug, real-time systems, reliability

- **S:** "In the WhatsApp platform, sometimes the delivery status for a message was lost."
 - **T:** "Make status updates reliable."
 - **A:** "I found that Exotel's delivery webhook sometimes arrived before my code finished saving the outgoing message, so the update found nothing. I changed the handler to upsert by the Exotel message ID. Now whichever arrives first creates the record, and the other updates it. This also made duplicate webhooks safe."
 - **R:** "No more lost statuses or duplicate messages."
-

Story 6 — Leading TorahAnytime

Use for: leadership, teamwork, mentoring, client handling

- **S:** "At Morpheme, I was given responsibility for TorahAnytime, a live platform for an Israeli client."
- **T:** "Manage development from our side and deliver features safely."
- **A:** "I broke client requests into tickets, assigned them based on each developer's strength, reviewed every pull request for logic, performance and security, merged and deployed to production, and verified

after deploy. When a developer got stuck, I paired with them. I also built features myself — likes, editable playlists, legacy speakers and the most-viewed filter."

- **R:** "Releases became regular and predictable, and the client trusted us with more work. I learned how to review code and communicate with an international client."
-

Story 7 — Saying no to a risky request

Use for: conflict, disagreement with manager, communication

- **S:** "There was a suggestion to automatically retry failed uploads to the dialer."
 - **T:** "Decide the right retry behaviour."
 - **A:** "I explained that if the first upload actually succeeded but the response failed, a retry would make the dialer call the same customer twice — a bad customer experience. I proposed logging failures and a manual re-trigger with a preview instead. I showed it with an example and documented the decision."
 - **R:** "The team agreed. No double calls to customers. I learned to disagree with data and offer an alternative, not just say no."
-

Story 8 — Learning something new quickly

Use for: adaptability, learning ability

- **S:** "At Morpheme I had to build serverless APIs on AWS Lambda with TypeScript, which I hadn't used before."
 - **T:** "Deliver working APIs within the sprint."
 - **A:** "I read the AWS docs, built a small test function first, learned serverless YAML, then built the CRUD APIs and a PDF-to-image function. I load tested with JMeter and optimised cold starts by reducing bundle size and reusing connections."
 - **R:** "Delivered on time, and later I optimised Lambda functions on FamilyOne too. Now AWS is one of my strengths."
-

Story 9 — Tight deadline

Use for: pressure, time management

- **S:** "Management needed consultation forms for 16 different diseases in the CRM, quickly."
 - **T:** "Deliver all forms in a short time."
 - **A:** "Instead of building 16 separate screens, I designed one dynamic form engine driven by JSON schemas, with draft autosave and pre-fill from the last consultation. Each new disease became a JSON file instead of new code."
 - **R:** "All 16 forms went live on time, and adding a new form now takes minutes."
-

Story 10 — Deployment issue on IIS

Use for: DevOps, production support

- **S:** "After deploying a config change on IIS, the app was still running old behaviour."
- **T:** "Find why the change wasn't applied."
- **A:** "I checked the iisnode logs and found that iisnode only restarts when watched files change, and my `src` files weren't in the watched list. I fixed the `watchedFiles` setting in `web.config`, added startup logs that print the loaded config, and documented the app-pool restart command."
- **R:** "Deployments became reliable, and anyone can now verify which config is loaded."

Quick mapping: question → which story

Question	Story
Biggest technical challenge?	1 or 2
Biggest achievement?	3
A production issue you fixed?	2 or 10
A mistake you made?	4
A difficult bug?	5
Leadership / mentoring?	6
Conflict or disagreement?	7
Learning a new technology fast?	8
Working under a deadline?	9
Improving performance?	1

Scenario questions ("What would you do if...")

Your API is suddenly slow in production. What do you do?

"First check monitoring and logs: which endpoint, since when, error rate. Check recent deployments. Then check the database — slow query log, execution plan, missing index, locks. Check CPU and memory of the server and whether the event loop is blocked. Check third-party API latency. Fix the root cause — often an index or a cache. If it's urgent, roll back the last deploy first. After that, write a short RCA."

Production is down and your manager is on leave.

"Inform the team and stakeholders immediately. Check health endpoints and logs. If a recent deploy caused it, roll back. Restart services if needed. Once it's stable, find the root cause, fix it properly, and share a short report."

You get a requirement that isn't clear.

"I ask questions before coding — expected input, output, edge cases, and who will use it. I write a short summary and confirm it with the stakeholder. In my CRM work I used screenshots of the old system to confirm behaviour."

You can't finish a task by the deadline.

"I inform my manager early, not on the last day. I explain what's done, what's left and why, and suggest options — reduce scope, extend the date, or get help."

A teammate's code has a bug that reached production.

"Fix the issue first. Then discuss it privately and politely, focus on the process, and add a test or checklist so it doesn't happen again. No blaming."

You disagree with a senior's technical decision.

"I share my concern with data and an alternative. If they still decide differently, I respect the decision and support it fully."

You find a security issue in existing code.

"Report it to my lead immediately, assess the impact, fix it with priority, and rotate any exposed secrets."

How do you estimate a task?

"Break it into small parts — API, DB, UI, testing, deployment. Estimate each, add a buffer for unknowns and integration, and share the estimate with the assumptions."

How do you handle a client who keeps changing requirements?

"Understand why it's changing, document every change, explain the impact on timeline, and agree on priorities. Short demos help catch changes early."

08 — Tricky Questions, Salary, Notice Period and Closing

Tricky questions test **honesty and calmness**. The rule: tell the truth briefly, stay positive, and end with what you learned or what you're doing now.

A. Job changes and experience

1. You changed 4 companies in under 4 years. Why?

"Each move had a clear reason and gave me more responsibility. Shaligram was my first job, and the role was changing to non-development work. At MSAI, the main project was handed over to the client and the team was released. At Morpheme I learned AWS and DevOps but worked on short client projects, and I wanted full ownership of a product, which I got at Atiya. Now I'm looking for a long-term role where I can grow in one place."

2. Why only about one year at each company? Will you leave us too?

"My early moves were about finding the right growth path, and some were outside my control, like the project handover at MSAI. Now I'm clear that I want to be a backend engineer in a strong product team, and this role fits that. I'm looking to stay long term and grow into a senior role here."

3. You joined Atiya in August 2025. Why are you leaving so soon?

"At Atiya I got full ownership and built 8 applications, which taught me a lot. But I'm mostly working alone on these codebases. I want a bigger engineering team with code reviews, system design discussions and products at larger scale. I'm not running away from anything — I'm looking for the next step in growth."

4. Your resume says 4 years. Isn't it about 3 years 8 months?

"Yes, exactly — I started in February 2023, so it's about 3 years and 8 months. I round it to almost 4 years." *(Always be exact if asked. Never argue.)*

5. Your title is MERN Stack Developer. Why apply for backend?

"My work has always been mostly backend — APIs, databases, queues, caching and deployment. I can build React screens, but the backend is where I add the most value, so I want to focus there."

6. Why didn't you get promoted?

"I moved for better responsibility instead of waiting for a title. In practice, I've done senior-level work — I led TorahAnytime with code reviews and production deployment, and at Atiya I own complete systems."

B. Questions that check if your resume is real

7. "Explain this resume line in detail." (any line) Be ready to explain every line with the 5-step formula from file 04. If one line is hard to explain, simplify it on the resume before applying.

8. "Did you really build all of this alone?"

"At Atiya, yes — the design, code and deployment are mine. I worked with business users for requirements and the IT team for server and database access. Most apps share the same layered template, and I used code generation for the 463 models, which made it faster. I can walk through any part."

9. "How did you measure 30% / 40% / 20% improvement?" - 30% API response time → "Measured average response time in logs and Postman before and after optimisation." - 40% deployment efficiency → "Manual deployment time and failure rate before vs after CI/CD." - 20% engagement → "Analytics numbers shared by the client after the redesign." *If you are not sure of a number, say "around" and explain how it was observed.*

10. "You wrote Kubernetes / Nest.js / Microservices. Have you used them in production?" Be honest about the level:

"Kubernetes — I used it on TorahAnytime for deployments, mainly working with existing configs, not designing the cluster. Nest.js — I know the structure (modules, controllers, providers, guards) and have built with it, but most of my production work is in Express." *(Adjust to the truth.)*

11. "Why isn't there a GitHub repo for these projects?"

"These are company and client projects, so the code is private. I can explain the architecture and code in detail, and I have personal projects on GitHub."

12. "Your GitHub commits show a different name/account." (if relevant)

"Those repositories are under the company's GitHub account. My commit identity there is the company-configured one."

C. Weakness and failure traps

13. "What is a skill you are weak at?"

"Large-scale distributed systems design — like Kafka at very high scale or designing multi-region systems. I understand the concepts and I'm practising system design, but I haven't designed one at that scale in production yet."

14. "What is something you'd do differently in your projects?"

"Start with TypeScript, write unit tests from day one, keep secrets out of the repo from the beginning, and set up CI/CD even for internal apps."

15. "Why is there no test coverage in your apps?"

"As a single developer under delivery pressure, I focused on validation, health checks, Postman collections and testing with real users. The CRM has Jest + Supertest. In a team I'd add service-level unit tests and contract tests."

16. "Why IIS and not Docker/cloud at Atiya?"

"It's the company's infrastructure standard — Windows Servers managed by IT with on-premise databases. I learned iisnode deeply. For the public WhatsApp app I used Cloudways with PM2. I've used Docker and AWS at Morpheme and I'm comfortable with them."

D. Salary and notice period

Before the interview, fill this:

Item	Your answer
Current CTC (fixed + variable)	_____ LPA
Expected CTC	_____ LPA (research Glassdoor / AmbitionBox / Naukri for "Node.js developer, 3–5 yrs, your city")
Minimum you will accept	_____ LPA (keep this private)
Notice period	_____ days (buyout possible? yes / no)
Earliest joining date	_____

"What is your current CTC?"

"My current CTC is [X] LPA." — Be honest. It's checked with payslips.

"What is your expected CTC?"

"Based on my experience in Node.js backend, the responsibility I've handled, and market range for this role, I'm expecting around [Y] LPA. I'm flexible depending on the overall package and growth opportunities."

"That's too high. Can you come down?"

"I understand. I'm more interested in the role and growth. Could you share the budget for this role? If the overall package and the growth path are good, I'm open to discussing."

"Why such a high hike?"

"My responsibilities have grown a lot — I own complete backend systems in production. The expected amount is in line with the market for this skill set and experience."

"What is your notice period? Can you join early?"

"My notice period is [N] days. I'll speak to my manager about an early release and can try to join sooner if needed."

E. Questions YOU should ask at the end (always ask 2–3)

Pick the ones that fit the round.

To the technical interviewer 1. "What does the backend architecture look like — monolith or microservices?" 2. "What does the deployment pipeline look like, and how often do you release?" 3. "How are code reviews done in the team?" 4. "What's the biggest technical challenge the team is working on right now?"

To the manager 5. "What would success look like for this role in the first 90 days?" 6. "How big is the team, and who would I work with most closely?" 7. "How do you support developers to grow into senior roles?"

To HR 8. "What are the next steps in the process?" 9. "What is the team culture like — remote, hybrid or office?"

Never ask in the first round: leave, salary hike cycle, "what does your company do?"

F. Closing the interview

Last line (say it with a smile):

"Thank you for your time. I really enjoyed this discussion, especially about [something they said]. I'm very interested in this role and I'm confident I can contribute to your backend team. I look forward to hearing from you."

After the interview: Send a short thank-you email or LinkedIn message within 24 hours.

Subject: Thank you — Node.js Backend Developer interview

Hi [Name], thank you for the interview today. I enjoyed learning about [team/product]. I'm excited about the opportunity to contribute to your backend team with my experience in Node.js, databases and production systems. Looking forward to the next steps. Regards, Sandeep Lodhi | +91 6266921342

09 — Last 30 Minutes Cheat Sheet

My one-liner

"Node.js backend developer, almost 4 years, Node + Express + MongoDB/SQL + Redis, healthcare CRM and WhatsApp automation at Atiya, AWS serverless and TorahAnytime lead at Morpheme."

Career in 4 lines

Company	Dates	One line	Why left
Shaligram Infotech	Feb 2023 – Aug 2023	JS, jQuery, MS SQL foundation; API +30%, SQL +20%	Role moving away from development
AB MSAI Research Labs	Aug 2023 – Oct 2024	Astrology platform R&D (70%), ThePNAC.com end to end	Project handed to client, team released
Morpheme Webnexus	Nov 2024 – Jul 2025	AWS Lambda, CI/CD, led TorahAnytime, built BikeShopShippers	Wanted full product ownership
Atiya Healthcare	Aug 2025 – now	CRM, WhatsApp platform, reports, automation; deploy on IIS/Cloudways	Want a bigger team and larger scale

Education

B.Tech CSE, TIT Bhopal (RGPV), 2019–2023, **CGPA 8.7**. 10th and 12th from Lalitpur, **76%** each.

Resume projects in one line each

- **TorahAnytime:** Israeli media + donation platform; lead: tickets, PR review, deploys; likes, playlists, most-viewed; Node, MySQL, Redis, Docker, AWS.
- **Hospital Appointment CRM:** ASP.NET → Node + React on SQL Server; 47 API modules; 463 generated models; Ameyo, WhatsApp, SMS; IIS; ~30% faster.
- **BikeShopShippers:** Medusa.js + PostgreSQL; UPS rates/labels/tracking; Stripe with webhooks; GitHub Actions CI/CD.
- **WhatsApp Platform:** Exotel webhooks → MongoDB → Socket.IO; multi-number routing; idempotent upsert by message ID; PM2 on Cloudways.

Numbers to drop

92 s → 300 ms · 45+ reports · 50M+ rows · 47 API modules · 463 models · 8 apps at Atiya · 16 dynamic forms · 30% faster processing · 40% faster deployments · 4 countries of clients

5 STAR stories (titles only)

1. Query 92 s → 300 ms (performance)

2. Event loop freeze in ViewReports (production issue)
3. 460+ table legacy migration (achievement)
4. Empty data from hourly ETL table (mistake learned)
5. Leading TorahAnytime (leadership)

Tech one-liners

- **Event loop:** sync code → microtasks (nextTick, promises) → macrotasks (timers, I/O).
- **Don't block the loop:** heavy CPU → worker thread or queue.
- **Layered API:** routes → validation → controller → service → repository.
- **Indexes:** ESR rule (Equality, Sort, Range); check with explain.
- **Keyset pagination:** `WHERE id > last ORDER BY id LIMIT n`.
- **Cache-aside:** read cache → miss → DB → set with TTL; fall back if Redis is down.
- **Retry:** only network / 5xx / 429, exponential backoff, never on 4xx.
- **Webhook:** public route, verify signature, respond fast, idempotent upsert.
- **JWT:** short access + long refresh; bcrypt passwords; RBAC middleware.
- **Scale Node:** stateless + more instances + load balancer + Redis + queues.
- **Socket.IO scale:** Redis adapter + sticky sessions.
- **Lambda cold start:** smaller bundle, reuse connections, provisioned concurrency.
- **Docker:** copy package.json first for layer cache; `npm ci --omit=dev`.
- **iisnode:** PORT is a named pipe — never parseInt.

Honesty rules

- Say exact experience if asked: about 3 years 8 months.
- Only claim what you can explain (WebRTC, Kubernetes, Nest.js — know your level).
- If you don't know: "I haven't used it in production, the closest I've done is..."

Questions to ask them

1. What does the backend architecture and deployment pipeline look like?
2. What would success look like in the first 90 days?
3. What's the biggest technical challenge the team is facing now?

Before you join the call

- [] Resume PDF open, same version you sent
- [] Water, charger, quiet room, camera at eye level
- [] Company research: product, customers, tech stack
- [] Current CTC, expected CTC and notice period ready
- [] Breathe slowly. Speak slowly. Smile.